

Projeto Programação Concorrente e Paralela - CUDA

Carlos Eduardo Nogueira Silva
Felipe Gomes da Silva
Gabriel Martins Brum
Luis Henrique Salomão Lobato

Novembro, 2025

1 Introdução

O mapeamento da propagação de doenças é um tema de grande relevância, principalmente após a pandemia de COVID-19. Através do conhecimento de como uma doença se transmite em uma população, é possível prever seu comportamento e tomar medidas preventivas.

Neste trabalho, é possível simular como que uma doença seria propagada com base na suas taxas de infecção, recuperação e mortalidade. Os casos de estudo são em mapas bidimensionais, com as seguintes dimensões: 20x20, 200x200 e 2000x2000.

Neste trabalho foram medidos os tempos de execução tanto em uma execução serial (CPU.c) quanto em uma execução paralela (GPU.c) utilizando CUDA.

2 Metodologia

2.1 Processamento

Para simularmos o espalhamento da doença, utilizamos uma matriz bidimensional onde cada célula representa um indivíduo. Cada indivíduo pode estar em um dos seguintes estados: Saudável (H - Healthy), Infectado (I - Infected), Ninguém (E - Empty) ou Morto (D - Dead). A simulação é realizada em passos de tempo discretos, onde em cada passo, o estado de cada indivíduo é atualizado com base nas regras de propagação da doença.

2.2 Implementação Serial (CPU)

A implementação serial processa cada célula da matriz sequencialmente, atualizando os estados com base nas regras de transição. Para cada iteração, o algoritmo verifica os vizinhos de cada célula saudável para determinar contaminação, e aplica probabilidades de cura (10%), permanência infectada (30%) ou morte (60%) para células infectadas.

2.3 Implementação Paralela (GPU com CUDA)

A implementação paralela utiliza CUDA para processar múltiplas células simultaneamente. Foi implementado o padrão *Grid-Stride Loop*, que permite que cada thread CUDA processe múltiplos elementos da matriz quando o número de threads é insuficiente para cobrir toda a matriz em uma única iteração.

Características da implementação GPU:

- **Kernel execute_iter**: Processa as transições de estado de forma paralela
- **Kernel verify_end**: Verifica condição de término (ausência de células infectadas)

- **Kernel setup_kernel**: Inicializa geradores de números aleatórios (cuRAND) para cada célula
- **Grid-Stride Loop**: Permite configurações flexíveis de threads/blocos

2.4 Configurações de Teste

Foram realizados testes com duas matrizes de entrada:

- **input_small.dat**: Matriz 32×32 (1.024 elementos)
- **realdata.dat**: Matriz 1024×1024 (1.048.576 elementos)

Para a implementação GPU, foram testadas 8 configurações diferentes de threads e blocos:

1. 1 thread em 1 bloco (processamento quase serial)
2. 1024 threads em 1 bloco
3. 512 threads em 2 blocos
4. 256 threads em 4 blocos
5. 128 threads em 8 blocos
6. 32 threads em 32 blocos
7. 64 threads/bloco em 16 blocos (1024 threads totais)
8. 512 threads/bloco em 2048 blocos (1.048.576 threads totais)

3 Resultados

3.1 Análise de Desempenho

Os tempos de execução obtidos demonstram claramente a vantagem do processamento paralelo em GPU, especialmente para matrizes maiores.

3.1.1 Matriz Pequena (32×32)

Configuração	Tempo (s)	Speedup
CPU (Serial)	0.000049697	1.0× (baseline)
GPU (1×1)	0.001918768	0.03× (mais lento)
GPU (1024×1)	0.000257147	0.19×
GPU (512×2)	0.000211140	0.24×
GPU (32×32)	0.000186943	0.27×
GPU (64×16)	0.000296315	0.17×

Tabela 1: Tempos de execução para matriz 32×32

Para a matriz pequena, observamos que a CPU supera a GPU devido ao *overhead* de transferência de dados entre host e device, além do custo de inicialização dos kernels CUDA. O tempo de setup do cuRAND e as transferências de memória (cerca de 4KB) dominam o tempo total de execução.

3.1.2 Matriz Grande (1024×1024)

Configuração	Tempo (s)	Speedup
CPU (Serial)	0.050861174	1.0× (baseline)
GPU (1×1)	2.939634447	0.02× (mais lento)
GPU (1024×1)	0.019354557	2.63×
GPU (512×2)	0.018144702	2.80×
GPU (256×4)	0.017437922	2.92×
GPU (128×8)	0.017076928	2.98×
GPU (1024×1024)	0.003021590	16.83×
GPU (512×2048)	0.002812409	18.08×

Tabela 2: Tempos de execução para matriz 1024×1024

Para a matriz grande, a GPU demonstra aceleração significativa:

- **Melhor resultado:** 512 threads/bloco × 2048 blocos alcançou **18.08× de speedup**
- **Pior caso GPU:** 1 thread × 1 bloco foi 57× mais lento que CPU (processamento essencialmente serial com overhead de GPU)
- **Tendência:** Quanto maior o paralelismo (mais threads/blocos), melhor o desempenho

3.2 Análise de Corretude

Os resultados de saída mostram consistência entre as execuções:

Matriz 32×32:

- CPU: 397-415 mortes, 66-73 sobreviventes (variação devido à aleatoriedade)
- GPU: 394 mortes, 74 sobreviventes (consistente entre todas as configurações)

Matriz 1024×1024:

- CPU: 415.429 mortes, 58.151 sobreviventes
- GPU: 415.363 mortes, 58.217 sobreviventes

A pequena diferença (66 mortes, 0.016%) é aceitável e atribuída à natureza estocástica da simulação e diferentes implementações de geradores de números aleatórios (rand() vs cuRAND).

3.3 Observações Importantes

1. **Grid-Stride Loop:** Permitiu que configurações com poucas threads (1×1 , 1024×1) ainda processassem toda a matriz corretamente, apenas de forma mais lenta.
2. **Ocupação da GPU:** Configurações com mais blocos (1024×1024 , 512×2048) maximizaram a utilização dos Streaming Multiprocessors (SMs) da GPU, resultando em melhor desempenho.
3. **Overhead vs. Computação:** Para problemas pequenos, o overhead de transferência domina; para grandes, o ganho computacional compensa amplamente.

4 Conclusão

Este trabalho demonstrou com sucesso a aplicação de programação paralela com CUDA para simular a propagação de doenças em populações bidimensionais. Os resultados evidenciam que:

1. **Escalabilidade:** O speedup da GPU aumenta proporcionalmente ao tamanho do problema. Para a matriz 1024×1024 , alcançamos até $18 \times$ de aceleração.
2. **Viabilidade:** Para problemas pequenos (32×32), a CPU serial permanece mais eficiente devido aos custos de overhead. Para problemas médios/grandes, a GPU torna-se indispensável.
3. **Flexibilidade:** O padrão Grid-Stride Loop permitiu que o código se adaptasse a diferentes configurações de hardware e casos de teste, desde 1 thread até mais de 1 milhão de threads.
4. **Corretude:** Os resultados entre CPU e GPU foram consistentes, validando a implementação paralela.

A experiência adquirida neste projeto ressalta a importância de escolher a arquitetura adequada (CPU vs GPU) baseada no tamanho do problema e nos requisitos de desempenho. Para simulações epidemiológicas em larga escala, a GPU demonstrou ser uma ferramenta poderosa e eficaz.

Trabalhos Futuros: Seria interessante explorar otimizações adicionais como uso de memória compartilhada (shared memory), operações assíncronas e análise de diferentes padrões de propagação de doenças (modelos SIR, SEIR, etc.).